QCon
全球软件开发大会

InfoQ 技术大会

从 Trace 到规模化实时评估

面向生产流量的 Agent 可观测实践

蚂蚁可观测团队

刘杨 / 架构师



QCon
全球软件开发大会

InfoQ 技术大会

目录

01 Agent 可观测面临的新问题

02 遵循 GenAI 语义约定的统一埋点与采集

03 以 Trace 为中心的端到端实时评估链路

04 规模化挑战：多阶段混合 AI 评估实践

05 真实业务落地案例与实践经验

06 从 Agent 执行到模型推理的深度分析

01

Agent 可观测面临的新问题

有了日志、指标和完整 Trace，仍判断不了 Agent 是否做对了任务

背景：长程 Agent 进入生产

Agent 开始在核心生产场景里跑长程任务，可观测体系却还是按微服务那一套设计的。



典型生产 Agent

个人办公 / 编程 Claude Code、Codex

长程、多步，自主规划并调用工具完成任务

C 端业务 健康问诊、灵光

面向海量真实用户，回答质量直接影响体验

传统可观测回答不了的三个问题

1. **Agent 是否正确完成了任务？**
   结果对不对，指标和 Trace 都不告诉你
2. **问题发生在哪个环节？**
   长程任务几十步，错误可能出在任何一步
3. **如何持续衡量和改善 Agent 质量？**
   需要在生产流量上持续度量、持续改进

挑战与解法

统一采集、实时评估、多阶段评估、深度分析，分别对应前面四个挑战，都已在蚂蚁生产环境中使用。

四个新挑战	对应解法
1 异构 Agent，数据语义不统一 同一个工具调用、RAG、记忆读写，各家表达各不相同	→ 统一埋点与采集 OTel GenAI 语义约定 · L1/L2/L3 三层 · 9 类 Span
2 质量难以度量，且必须在线上度量 没法用代码判对错；上线前测试覆盖不了	→ 以 Trace 为中心的实时评估链路 声明式表达式接入 · 分钟级端到端延迟
3 规模化评估的 token 成本 百万级流量，不可能每条都交给大模型	→ 多阶段混合 AI 评估 小模型 / 正则全量粗筛 + 大模型按需精判
4 推理引擎盲区 性能慢在哪个 token、答偏从哪个 token 开始，都在黑盒里	→ 从 Agent 执行到模型推理 关联推理引擎 · Token 粒度定位 · 概率分布评判稳定性

分钟级	90%+	周级 → 天级
端到端评估延迟	token 成本降低	新 Agent 接入周期

02

遵循 GenAI 语义约定的统一埋点与采集

统一模型调用、工具调用与执行过程的遥测语义，并支持业务扩展

挑战：异构 Agent，缺乏统一的数据语义

同样的工具调用、模型调用、RAG 和记忆读写，在不同 Agent 里是完全不同的数据模型。

语义	Agent 框架 A	Agent 框架 B	自研业务 Agent 示意
工具调用	独立 Span	父 Span 上的 event	日志行
模型输入	messages 数组	拼接后的 prompt 字符串	不上报
记忆读写	无	自定义 tool	RPC 调用
任务状态	无	session 字段	业务表

三个影响

采集接入覆盖难 → 解析与关联难 → 统一评估难

目标：统一的数据表达
同时保留各 Agent 特有信息

框架、运行机制、业务场景各不相同，数据格式、字段定义和轨迹的组织方式也就各不相同。

Agent 可观测轨迹埋点规范

基于 OpenTelemetry GenAI 语义约定扩展出三层规范，覆盖 9 类 Span，不同 Agent 的数据可以放在一起比较。

分层设计：通用性与业务灵活性兼顾

L3 业务自定义 各业务特有信息 — ant.*
ant.biz.scene · ant.exp.group ...

L2 场景化 Agent 执行过程语义 — agentic.*
agentic.turn.number · agentic.tool.count ...

L1 通用 模型 / 工具调用的通用字段 — gen_ai.*
gen_ai.operation.name · gen_ai.tool.name ...

基础：OpenTelemetry GenAI 语义约定

越往上越贴近业务，越往下越通用

覆盖 9 种 Span 类型

模型与编排	AGENT · CHAT · LLM
工具	TOOL · MCP
检索增强	RETRIEVER · RERANKER · EMBEDDING
安全护栏	GUARDRAIL

一次 Agent 执行的每一步，都落到一种标准 Span 上

Agentic 专属属性

agentic.turn.number 第几轮 · agentic.tool.count 工具调用数

gen_ai.tool.type 枚举：function · extension · datastore · + skill 新增

属性前缀白名单门禁

保障数据一致性

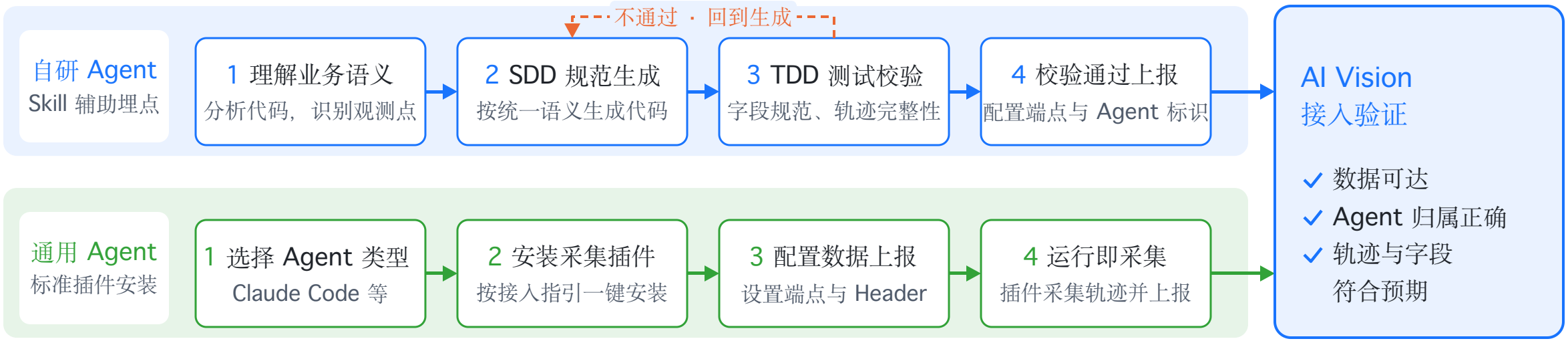
上报的 Span 属性 → 前缀校验 白名单

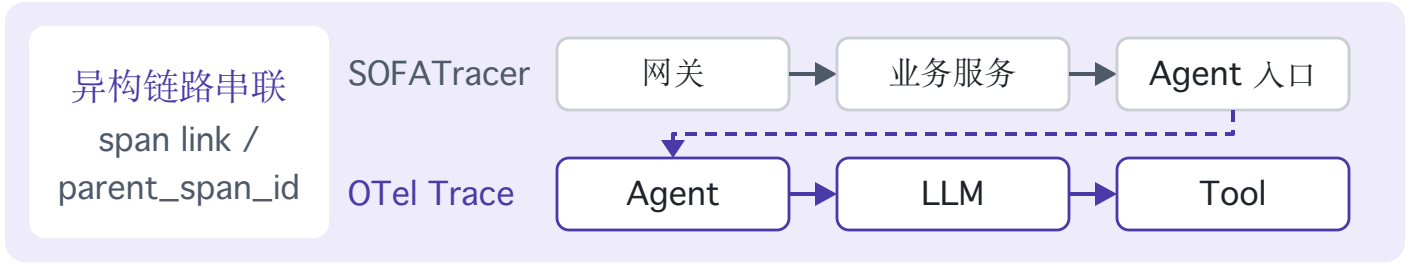
- gen_ai.* · agentic.* · ant.* ✓ 通过
- my_attr.foo × 不符合规范前缀

→ 不同 Agent 数据横向可比
支撑轨迹洞察、异常检测
与统一评估

自动化规范埋点：如何快速接入

自研 / 通用两条采集路径，统一在 AI Vision 验证；新业务 Agent 接入从周级缩短到天级。





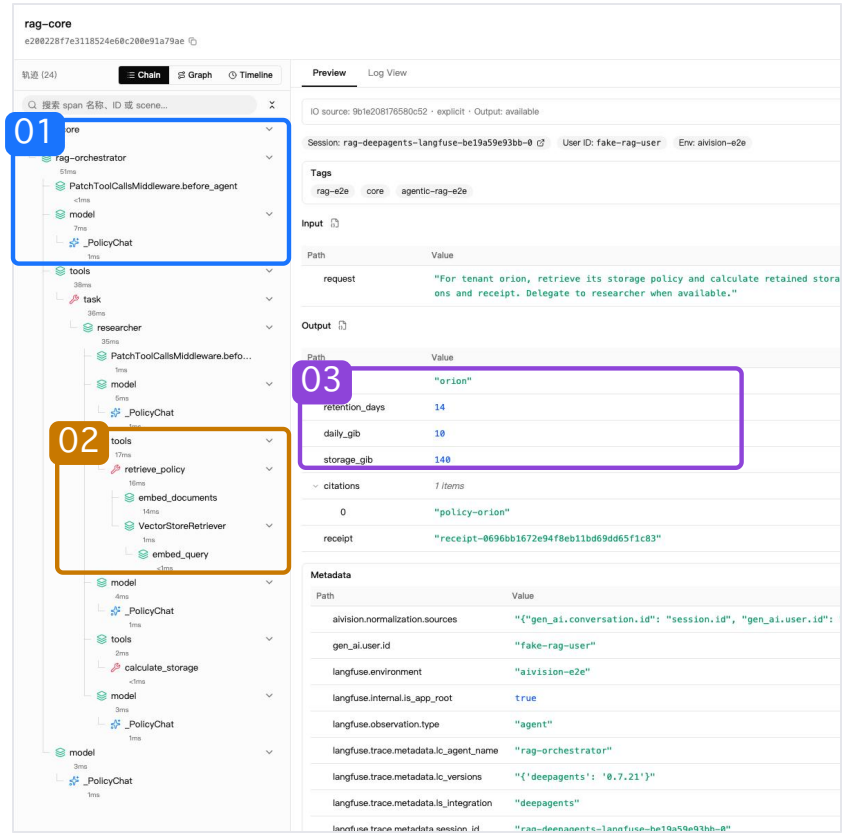
接入周期 周级 → 天级

覆盖 100+ 种 Agent 业务

含公司内部主流 Coding Agent 与 C 端 Agent 等项目

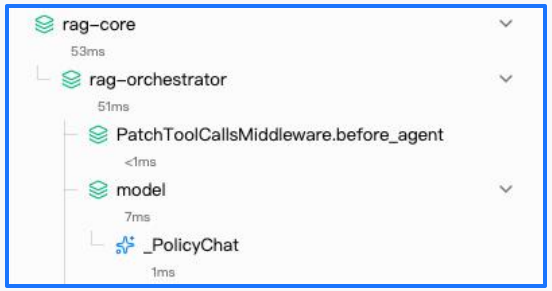
运行轨迹 · Chain 视图

主任务、子 Agent 和工具按层级展开，每一步由谁发起一目了然，还能对照输入和结果。



同一份轨迹还可切换 Graph / Timeline 视图，
点击节点查看详情。

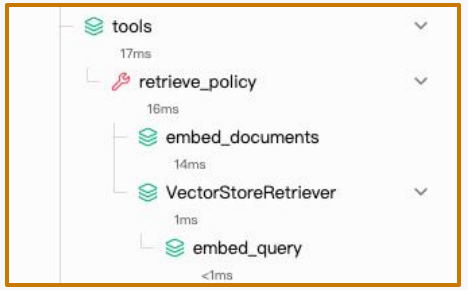
01



按层级还原执行

按层级展开主任务、模型调用、
工具和子 Agent。

02



下钻到具体工具

检索、向量处理、计算等工具调用，
都挂在所属任务下。

03

tenant	"orion"
retention_days	14
daily_gib	10
storage_gib	140

核对输入与结果

在同一面板里查看请求、
结构化结果和引用。

03

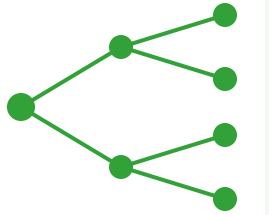
以 Trace 为中心的端到端实时评估链路

Span 聚合、维度提取、任务筛选、证据提取、评估执行与结果沉淀，声明式适配异构 Agent

为什么在线评估，为什么以 Trace 为中心

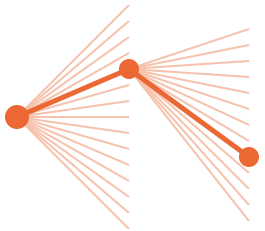
上线前测试一定会漏，质量只能在线上实时度量；一次请求天然就是一条 Trace，以它为中心，关联和维度问题一并解决。

为什么要在线评估



传统应用：决策 = 代码分支

- 输入受限，分支可以枚举
- 上线前测试能覆盖所有路径



Agent：决策 = 每一个 token

- 输入开放：用户可以说任意一段话
- 每步十万级候选，按概率采样

上线前测试一定会漏

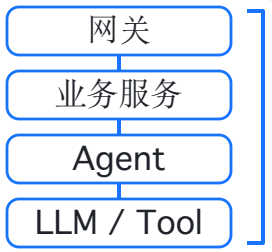
要在生产流量上实时评估完整轨迹：结果 + 过程

为什么以 Trace 为中心



传统监控：按节点采日志

- 日志分散，指标之间难关联
- 下游拿不到上游的业务维度



Trace：一次请求天然一条链

- 跨节点天然关联，维度沿链继承
- 服务指标与效果指标在同一条链上算

以 Trace 为中心加工实时数据

关联与维度问题一并解决，服务与效果指标同链计算

整体架构：分钟级端到端实时评估

采集 → 六个环节的实时流水线 → 看板、告警、下钻与业务决策；维度前置圈选、后置分析，评估器按任务调用。

AI Vision · 面向 Agent 的实时轨迹评估平台

端到端延迟：分钟级

采集与存储

Agent
上报轨迹

AntCollector
采集

存储

Stargazer

CeresDB

实时流水线

任务调度　　评估运行

01 Span 聚合	02 维度提取	03 任务筛选	04 证据提取	05 评估执行	06 结果沉淀
长程 Trace 何时算完整?	按哪些维度 观测分析?	哪些 Trace 跑哪些评估器?	评估器需要 哪些证据?	怎么判定?	结果怎么用?
完整性判定 迟到、乱序处理	已有标签直取 语义模型打标	按维度圈选 定向观测	只取必要材料 省 token	证据 + 判定标准 → 标签 + 原因	多维质量指标 + badcase 明细

前置：圈选　　后置：多维指标与下钻

评估器库　被评估运行按任务调用

评估粒度：Span 单步 · Trace 整条轨迹 · Session 多轮会话

Code 规则　　LLM 评判　　Skill　　Agent

可观测与决策

可观测

- 质量看板
- 实时告警
- 多维下钻归因

业务决策

- 变更准出
- 实验推全

声明式表达式：低成本适配异构 Agent

评估器只声明需要哪些变量，DSL 表达式负责从不同结构的 Trace 中取值；接入新 Agent 只改表达式，不改埋点。

① 业务 Agent 上报的 Trace

- AGENT assistant_agent
  - CHAT 对话历史 / 用户输入
  - …
  - TOOL memory_read
  - LLM 生成回答
  - TOOL memory_write

不同 Agent 的框架、字段、层级各异
埋点不规范也能低成本接入

② DSL 表达式取变量

示意

内置变量 · 开箱即用

user_query　history

业务自定义变量 · 用表达式声明

old_memory　读到的旧记忆

```
spans[name=="memory_read"].output
```

new_memory　本轮写入的新记忆

```
spans[name=="memory_write"].input
```

换一个 Agent，Span 名或字段不同，
只需改这两行表达式，评估器不用动

③ 记忆更新评估器

判断：新记忆是否违背用户陈述的事实

prompt 中直接引用变量

{{user_query}}　{{history}}

{{old_memory}}　{{new_memory}}

例 user_query：
“我上个月已经搬到上海了”
old_memory：常住城市：北京
new_memory：常住城市：北京
✗ bad：应更新为“上海”

输出：bad / good + 判定原因

零埋点改动
新增评估场景不改埋点

只改表达式
适配新 Agent 的 Trace 结构

容忍不规范
埋点不规范也能取到变量

维度富化（规划中）
模型打标：对话→话题分类

04

规模化挑战：多阶段混合 AI 评估实践

小模型 / 正则全量、低成本、高召回粗筛，大模型按需加载上下文精准裁决

规模化挑战：评估成本

既要覆盖最终回答与关键决策节点，也要有足够的评估量支撑效果比较；全部交给大模型，成本扛不住。

覆盖度　最终回答效果 + 轨迹中关键决策节点的正确性

充分度　对足够量级的真实流量进行评估，每个实验、每个分组都要足量



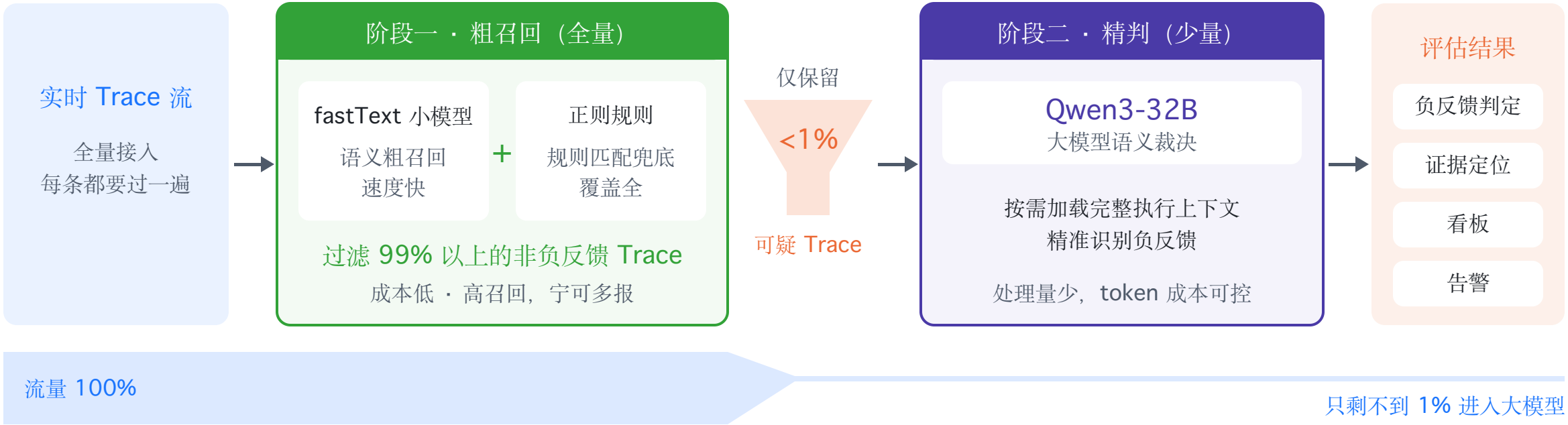
10	5	4	1 万	200 万
评估器	A/B 实验	每实验分组	每组样本	次评估

规模推演：输入越长，Token 消耗越大；底线场景要全流量评估，评估规模还会随业务流量增长。

工程问题：在保证覆盖和评估量的前提下，减少高成本大模型评估的处理规模

多阶段评估：小模型粗筛，大模型精判

解法是先粗召回、再精判。以负反馈检测为例：小模型 + 正则在第一阶段过滤 99% 的流量，只有 1% 进入大模型。



99% 流量在第一阶段被过滤
小模型 + 正则
全量、低成本、高召回

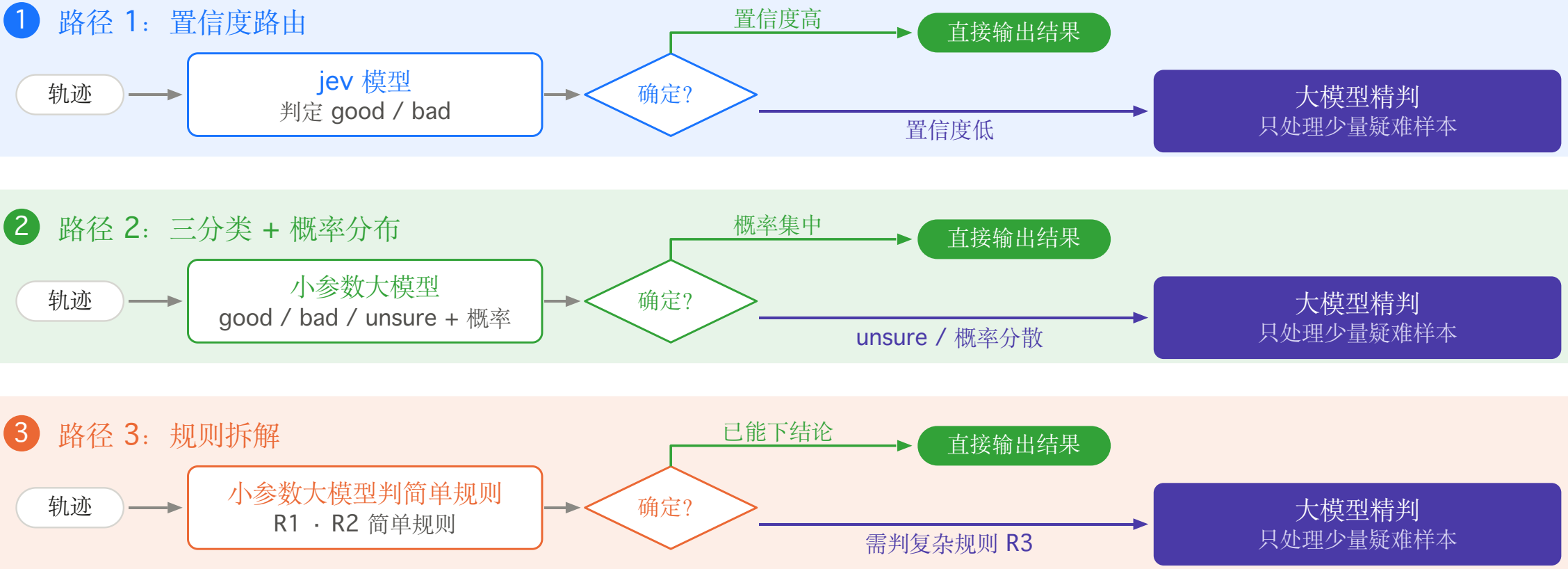
1% 进入大模型精判
按需加载完整执行上下文
精准语义裁决

↓99% token 成本
相比全部交给大模型评估

从专用小模型到通用方案

每个问题都训练一个小模型成本太高，我们在探索三条通用路径：置信度路由、三分类概率、规则拆解。

共同思路：便宜的模型先判，不确定的再升级给大模型



05

真实业务落地案例与实践经验

以某健康问诊 Agent 为例：把典型问题变成指标，再用指标驱动质量改进

生产案例：把典型问题变成指标，再用指标驱动改进

以“药品问题未查权威知识库就回答”为例：先把典型问题变成可度量的指标，再用指标做归因和实验。

把典型问题变成可度量的指标

① 典型问题

业务方归纳高频 badcase，明确每类问题要评估什么

本例

药品疾病类问题未查权威知识库就回答

用到　问题归类

② 造评估器

定义证据、边界与判定步骤，在数据集上迭代到正确率达标

本例

med_search 应调未调评估器

用到　证据提取、Skill

③ 定向观测

用语义标签、实验分组圈选流量，只在目标流量上跑

本例

问题类型 = 药品疾病，只评估这部分流量

用到　维度提取、多阶段

指标

评估器

badcase 率

本例

med_search 应调未调

用指标持续驱动质量改进

用法一 指标驱动归因

badcase 率 + 关键节点上的评估器，自动定位是哪个决策节点出了错

本例　这类问题归因到工具调用节点

用到　Trace 中心评估链路

用法二 指标度量实验效果

分组比对 badcase 率，定位劣化 → 优化 → 再上线验证

案例　OCR 截断寻优；新模型 Skill 回归

用到　多维指标、下钻

优化上线后回到 ①：持续看 badcase 率

从典型问题到评估器

每一类典型问题对应一个评估器；最后一行是本章贯穿的例子，对应“med_search 应调未调”。

典型问题（示意）	问题归类	评估器
替家人问诊，回答成了提问者本人	判错问诊人（常见代爸妈问诊）	问诊主体错判
前几天聊过的病情，这次没用上	没记住几天前讨论的病情	个人记忆消费
被问“你是谁”时回答混乱	自我身份认知错误	助手身份认知
反复追问已经给出的信息	重复或不合理的追问	追问合理性
中文回答里夹杂英文	透出不合理英文（老年用户难理解）	回答中透出不合理英文
内部标签直接出现在回答里	泄露内部 prompt 内容	指令泄露
药品疾病问题凭记忆作答	药品疾病类问题未查权威知识库	med_search 工具应调未调

评估器定义：证据、边界、判定

以“med_search 应调未调”为例：能上生产的评估器要只给必要证据、划清边界、写出可复核的判定步骤。

输入：只给必要证据

- 历史上下文
- 用户输入
- 工具列表
- Agent 输出

评估器：med_search 应调未调　　定义节选 · 已脱敏

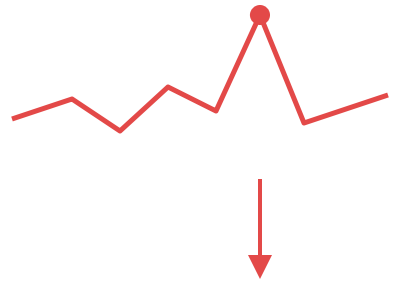
1. 任务与证据
   - 任务：判断是否应调用 med_search 而未调用
   - 证据：只看左侧四类输入，不看整条 Trace
2. 应调 / 不应调的边界
   - 必须调用：询问药品用法用量、疾病诊疗方案
   - 无需调用：寒暄、闲聊、追问已检索过的内容
3. 判定步骤与结论标准
   - 应调用而未调用 → bad，写明缺失的检索意图
   - 其余情况 → good

输出

- good / bad 标签
- 判定原因
- 子标签（可选）

子标签表达 badcase 子分类，可自定义

badcase 率看板



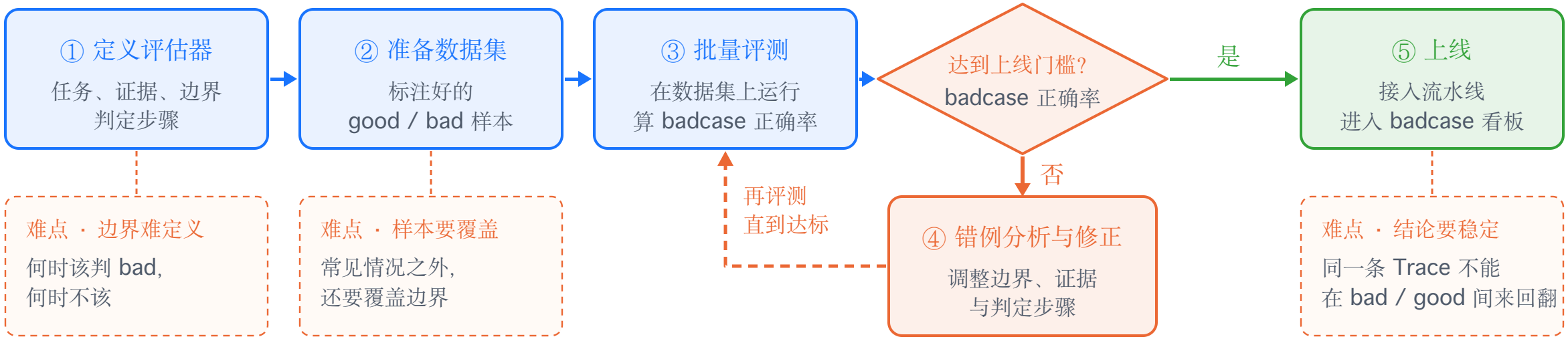
下钻到具体 badcase

为什么只给必要证据

- 准确　减少无关材料干扰
- 省钱　输入 token 更少
- 更快　评估性能更高

评估器开发：Skill + CLI 迭代

定义写完还不够：要在数据集上经过多轮“评测 → 错例分析 → 修正”，badcase 正确率达标才能上线。



我们提供

Skill + CLI

评估器开发 Skill

方法与步骤沉淀下来，交给 Agent 执行

定义 | 评测 | 错例修正

配套 CLI

命令行完成，便于反复迭代

数据集评测 | 错例查看 | 上线发布

门槛前置

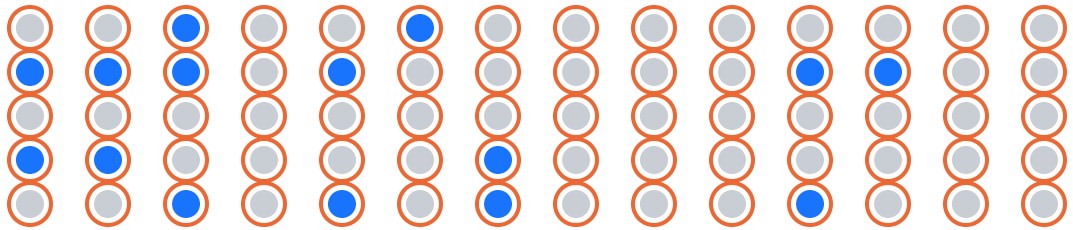
正确率不达标的
评估器不上线

定向观测：精准圈选流量

med_search 只该评估药品疾病类问题，但 Trace 上没有这个标签；语义打标后圈得准，无效评估和 token 成本一起降。

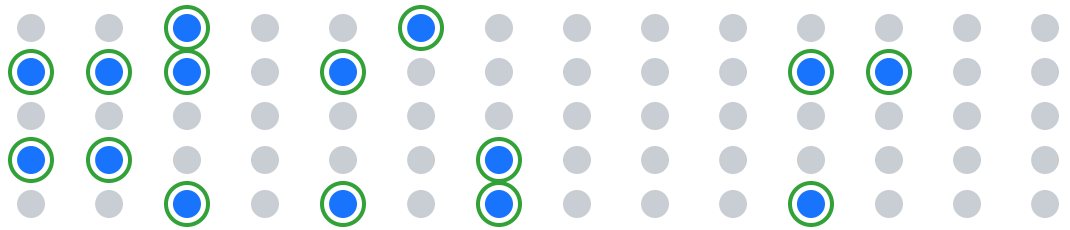
例：med_search 应调未调评估器（示意）

没有标签：只能在全部“健康问答”上跑



大量非药品疾病类的 Trace 也被评估：
token 浪费，badcase 率还被稀释

有标签：圈选“问题类型 = 药品疾病”再跑



只评估相关 Trace，
每一次评估都是有效的

● 药品疾病类 Trace　● 其他 Trace　◯ 被评估

标签从哪里来

Trace 上已有的标签 → 直接提取

业务 · 场景 · 渠道 · 实验分组 · 模型版本

→

需要语义理解的标签 → 模型打标

问题类型 · 疾病类型（如糖尿病）……

→

沉淀为维度

参与圈选分组（定向观测）+ 多维指标下钻

指标驱动归因

评估器跑起来就有了指标：线上自动把 badcase 归因到出错节点，算出每类问题占多少，决定先修哪个。

质量信号

负反馈 badcase 率
评估器产出 · 实时统计

业务质量指标
业务侧大盘
异动触发告警

Agent 执行轨迹（多步决策）



在关键决策节点上定义评估器，评估常见错误

关注：负反馈 badcase 率是否上升？根因是什么？

动线 A

统计关联
从整体比例下钻

负反馈 badcase 率 异常上升 → 对比各节点 效果指标时序 → 找出同期异动 的节点指标 → 统计关联 定位异常节点

动线 B

逐条归因
从单条 badcase 下钻

单条 badcase 进入归因 → 遍历节点评估器 逐个判定 → 命中异常节点 定位出错步骤 → 批量 badcase 根因统计聚类

例：药品问题未查知识库、凭记忆作答 → 命中工具调用节点：med_search 应调未调

指标度量实验效果：OCR 参数寻优

实验分组 + 评估器，比对各组 badcase 率：OCR 截断参数里，“历史 2000 / 当前轮 4000”最低（0.0273%）。

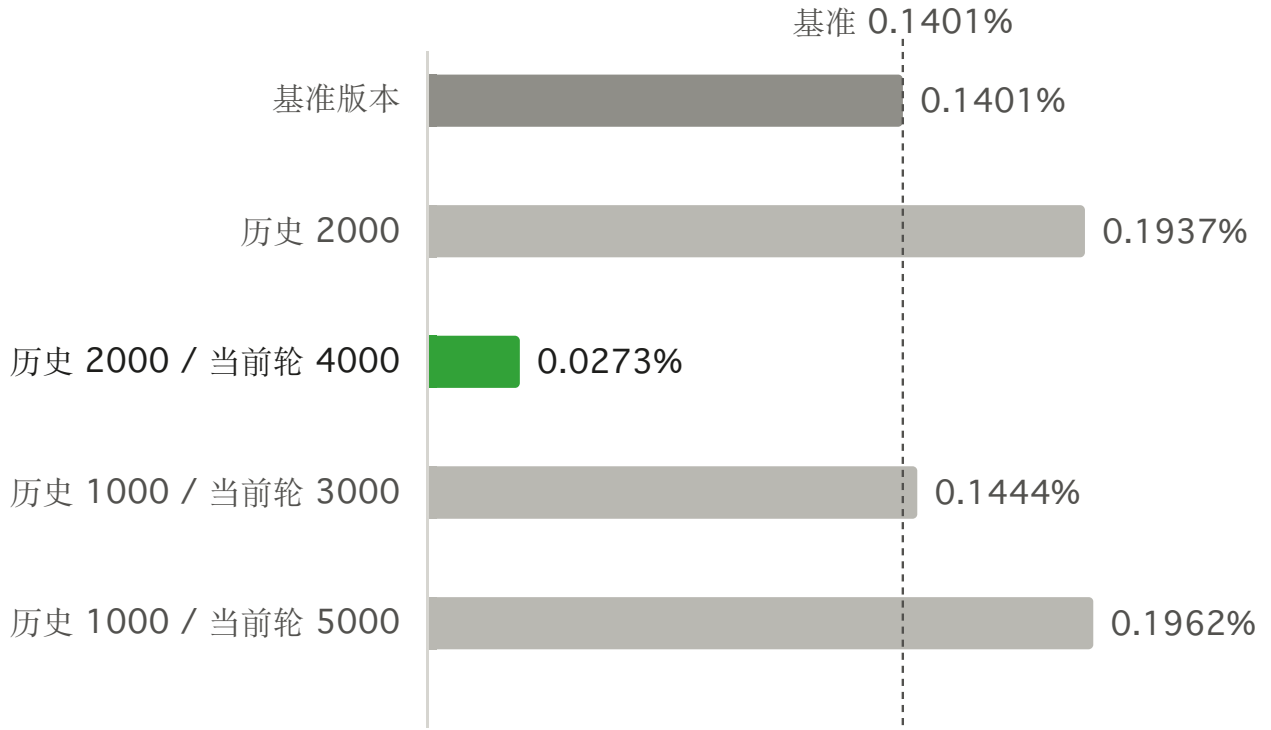
流量圈选与分组



案例 1：OCR 截断长度寻优

OCR 把图片解析成文字，太长需要截断，
历史上下文与当前轮各一个阈值，
哪组参数 badcase 率最低？

各实验分组 badcase 率



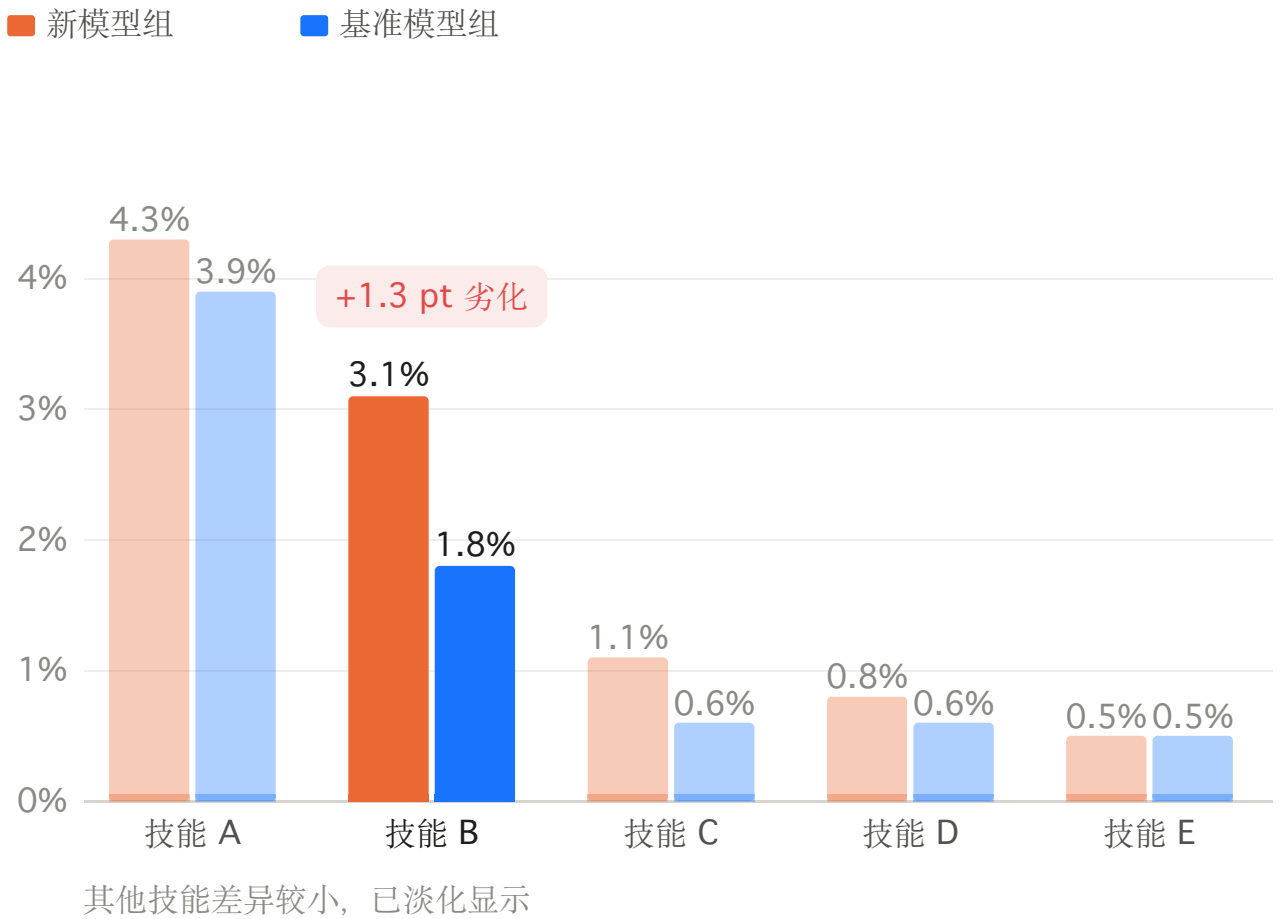
0.0273%

最合适配置：历史阈值 2000 + 当前轮 4000
约为基准的 1/5；整体 0.1399%

指标度量实验效果：模型升级回归

同样的用法，换成 Skill 漏召回评估器：新模型组技能 B 漏召 3.1%，高于基准组的 1.8%，优化后再上线验证。

Skill 漏召回评估器 badcase 率：新模型 vs 基准模型



其他技能差异较小，已淡化显示

推全前发现劣化，下一步：优化闭环

1. **定向评估发现劣化**
   技能 B 漏召 1.8% → 3.1%
2. **针对性优化**
   适配 prompt / SFT / RL
3. **新模型再次上线实验**
   同样做分组定向评估
4. **再验证**
   技能 B 是否恢复、其他技能是否劣化

06

从 Agent 执行到模型推理的深度分析

关联推理引擎，在 Token 粒度定位性能问题；从概率分布评判评估的稳定性

顺着 traceId，把一次 LLM 调用展开到 Token

Agent Trace 追到 LLM Span 只剩输入、输出和耗时，引擎内部是黑盒。我们在引擎内逐 token 记录耗时、并发和 Top-k 候选概率。

逐 token 记录：耗时 + 候选概率

一次回答 = 一串 token，每个都记下：　　示意

token	耗时	Top-3 候选概率（橙色 = 采样结果）		
一般	28 ms	一般 0.71	会 0.18	可能 0.06
不	31 ms	不 0.88	很少 0.07	会 0.03
伤	30 ms	伤 0.95	损 0.03	影响 0.01
肾	29 ms	肾 0.99	肝 0.01	。 0.00

耗时 → 定位慢在哪个 token

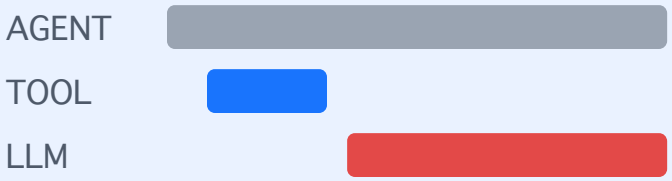
候选概率 → 看从哪个 token 开始偏、采样犹不犹豫

① Agent Trace 发现异常

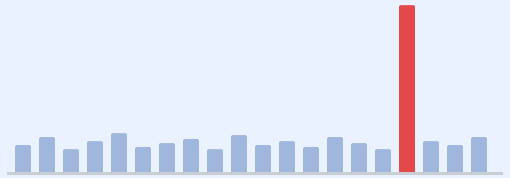
② 同一 traceId 下探到 Token

③ 定位根因

性能案例：这一步为什么慢？



这次 LLM 调用输出明显卡顿



第 125 个 token 卡了 6.8 s

每根柱 = 一个 token 的耗时



同实例别人的长 prefill 插队

底座：以 Token 为中心的 Trace · 三大引擎内直接埋点 · 全量不采样 · 观测开销 ‰ 级 · 定位效率 10x+

从概率分布评判评估的稳定性

固定 badcase 集重跑，结果偏差接近 10%。我们直接透出评估器标签 token 的概率，低置信结果由业务决定丢弃还是复核，不用多跑几次投票。

现象

~10%

固定 badcase 集，同 prompt 同模型重跑的偏差

第 1 次 bad　第 2 次 bad

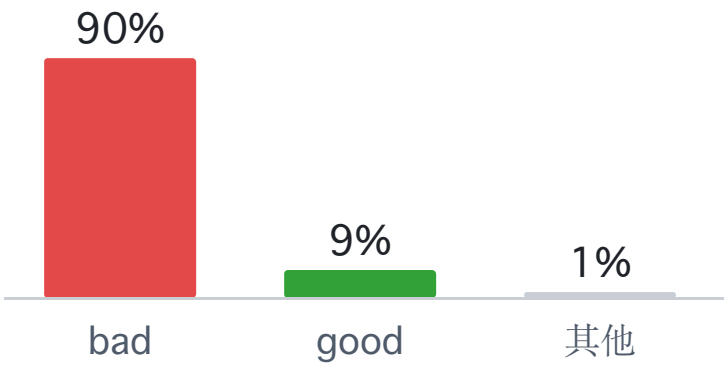
第 3 次 good　第 4 次 bad

同一条 trace，标签来回翻

映射到 token 概率

评估器输出的第一个关键 token 就是标签

{"label": "　?　", "reason": ...



解法：一次推理拿到置信度

读取标签 token 的概率（logprobs）

高置信　p ≥ 阈值

直接采用

低置信　p < 阈值

丢弃 / 人工复核 / 大模型精判

同一个 token 概率信号，可以用在两处

被观测 Agent　探索中

质量预测 · 聪明采样

低置信 / 高熵片段优先评估

Token 概率分布

候选概率 · 熵 · 采样参数

由引擎显微镜采集

评估器　已落地

标签置信度

低置信结果复核或升级精判

总结

轨迹流量、实时评估、质量指标、告警、多维分析、badcase 下钻归因已经串成一条链路，结论再回到优化。



我们做了什么

02 统一语义采集
OTel GenAI 语义约定 + 自动化埋点

03 声明式实时评估
以 Trace 为中心，分钟级端到端

04 多阶段评估
小模型粗筛 + 大模型精判，大幅降本

05 生产落地
典型问题变指标，指标驱动归因与实验

06 跨层下探推理引擎
Agent Trace 关联到 Token 粒度

开源计划

蚂蚁可观测团队联合 OceanBase 开源团队，分阶段开放观测与评估能力。

计划

10 月 31 日	11 月 30 日	2027
AI Vision 0.1	AI Vision 0.2	AI Vision 1.0

基础观测

- 数据采集
- 轨迹 / 会话查询与展示
- 指标统计与看板

对应本次分享 02 统一埋点与采集

核心评估

- LLM as Judge
- 评估数据集与任务
- Badcase 展示与分析

对应本次分享 03 实时评估 · 04 多阶段评估

分析增强

- 分组评估与比对
- 归因、聚类与多维下钻
- Skill / Agent 评估器
- 会话级评估

对应本次分享 06 分组评估 · 下钻 · 评估器

分阶段开放前后端核心模块，围绕开箱即用轨迹接入、运行看板、实时评估与使用案例开展社区共建

AiCon 北京站

全球人工智能开发与应用大会

探索 AI 应用边界

Explore the limits of AI applications

2026 年 12 月 18–19 日 / 北京新云南皇冠假日酒店

参会咨询

专题：Post-Training Engineering | 数据、奖励与模型能力塑造



专题出品人：余瑶

飞猪 / 资深算法专家

专题：大模型推理与算力调度实践



专题出品人：范苑

阿里云 / 专有云计算产品负责人

专题：Knowledge Engineering | 让 Agent 理解企业



专题出品人：盛茂家

火山引擎 / Viking 团队技术负责人

专题：Loop & Graph Engineering



专题出品人：孙子荀

腾讯 / 腾讯高级总监

专题：AI 原生架构的演进与治理



专题出品人：刘中兵

快手 / 架构治理团队 & AI 架构演进负责人

专题：Agentic Software Engineering | 从代码生成到软件交付



专题出品人：杨杰

荣耀 / 首席 IT 技术架构师

QCon
全球软件开发大会

InfoQ 技术大会

THANKS

Q&A

从 Trace 到规模化实时评估 · 蚂蚁可观测团队

